Aiding Software Root Cause Analysis with Large Language Models

- Evaluation of the Effectiveness of Fine-tuned T5, GPT, and RAG in the handling Customer Fault Reports

Shijun Feng
Ericsson AB
shijun.feng@ericsson.com

Department of Computer Science
XYZ Institute
jane@xyz.edu

***Abstract*—Software systems generate a substantial number of fault reports during pre-deployment customer testing, making manual root cause analysis (RCA) both time-consuming and error-prone. This study explores the use of large language models (LLMs)—specifically T5, GPT-2, and a retrieval-augmented generation (RAG) model—to automate and enhance the RCA process in a domain-specific software engineering setting. Using a curated dataset of real-world fault descriptions and resolutions, the models were fine-tuned and evaluated using BLEU-4, ROUGE, and BERT-based semantic similarity metrics. Results indicate that T5 outperforms GPT-2 in lexical and structural fidelity (e.g., BLEU-4: 0.1810 vs. 0.1210), while RAG achieves the highest semantic similarity (BERT score: 0.7715). These findings suggest that combining T5's precision in technical phrasing with RAG's contextual understanding may offer a promising direction for developing intelligent RCA assistance tools that improve both accuracy and relevance in software fault diagnosis. Future work will focus on hybrid model optimization and user-centered system integration for real-world engineering workflows.**

***Index Terms*—Large Language Models, Root Cause Analysis, Software Fault Diagnosis, Artificial Intelligence, Automation**

I. Introduction

A. Context and Problem Statement

The deployment of software solutions in complex cloud-native environments generates a high volume of operational trouble reports (fault reports). Efficient handling of these reports, particularly in performing Root Cause Analysis (RCA), is critical for maintaining system reliability and customer satisfaction. Currently, fault report management is largely manual and experience-driven. A key challenge is the management of duplicate reports, which necessitates developers to manually verify, classify, and link issues [1]. This leads to significant redundant work, delayed response times, inconsistent fault handling, and ultimately, operational inefficiencies that impact service quality. This highlights the urgent need for an automated, data-driven approach to RCA.

II. Related Work

1) Traditional ML Approaches in RCA: Root cause analysis (RCA) using machine learning has gained significant traction across domains such as software engineering, manufacturing, and networking. Traditional approaches often apply supervised models like support vector machines (SVMs), decision trees, or Naïve Bayes classifiers to classify faults based on historical data.

Wang et al. [2] train SVMs on textual bug reports to predict fault types (e.g., concurrency, memory, semantic bugs), achieving solid accuracy but providing no interpretability or semantic insight to support engineers during resolution. Chen et al. [3] combine PCA and SVM for fault detection in manufacturing, successfully identifying key structured features but excluding natural language inputs. Similarly, Zhang et al. [4] apply ML to SDN faults using time-series data, offering fast classification but limited diagnostic support.

Among interpretable models, Jovic et al. [5] use Naïve Bayes to identify predictive terms in bug descriptions, offering limited lexical guidance to engineers. However, none of these works integrate semantic retrieval or lexical pattern analysis to directly aid fault resolution.

2) Neural Network and Deep Learning Approaches in RCA: Deep learning has been increasingly adopted for root cause analysis (RCA), particularly in domains like manufacturing, network management, and software systems. Unlike traditional machine learning approaches, deep neural networks offer powerful feature learning capabilities, but not all methods provide semantic insight or lexical interpretability needed to support engineers in resolving issues.

Artificial neural networks (ANNs) and residual causal models like Sparse Causal Residual Neural Networks (SCRNN) have been applied to structured sensor data in manufacturing and quality control tasks [6]. These models perform well in predicting fault contributors from numerical logs but do not operate on natural language input or offer semantic interpretability.

Other approaches leverage deep learning architectures such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), autoencoders, and transformers for unsupervised anomaly detection in system logs [7]. While these methods are useful for identifying outliers, they typically lack causal reasoning and are not designed to interpret textual summaries or aid fault resolution.

More promising results are seen in graph-based models. KGroot [8], a recent framework combining knowledge graphs with graph convolutional networks (GCNs), constructs semantic fault graphs from event traces and textual

logs. It enables similarity-based retrieval of past faults with known causes, providing contextual and semantic guidance. This makes it particularly well-suited for cases where fault descriptions, priorities, and summaries are available.

Another technique involves combining LSTM networks with Bayesian neural networks (BNNs) to model probabilistic causality across time-series data [9]. Though effective in capturing causal structures, it relies entirely on structured logs and does not process natural language input.

Summary of Related Word: Traditional machine learning and deep learning methods have laid the foundation for automated root cause analysis, especially in classification and clustering tasks. However, these approaches often fall short in dealing with unstructured, domain-specific text where lexical and semantic nuance is crucial. Recent work using graph-based or retrieval-augmented models has improved case similarity detection, yet lacks generalization across diverse linguistic expressions. This opens the door for large language models (LLMs), which inherently support contextual understanding, semantic retrieval, and natural language generation. As such, this thesis explores whether LLMs—specifically T5, GPT-2, and RAG—can serve not as replacements but as intelligent assistants to engineers performing RCA, addressing both relevance and clarity in fault resolution.

III. METHODOLOGY

This section details the research methodology, experimental setup, data preparation, and evaluation procedures used to compare the performance of Large Language Models (LLMs) in generating Root Cause Analysis (RCA) explanations.

A. Research Approach and Scope

The study employs a **quantitative, experimental, and comparative** methodology rooted in a positivist paradigm. The objective is the objective, measurable, and replicable evaluation of different LLM architectures for RCA tasks. We utilize an experimental design to assess model outputs under controlled conditions using standardized, automated metrics (`BLEU`, `ROUGE`, `BERTScore`).

The research is strictly **limited to textual input features** (Summary, Description, Priority, Answer) and **automated evaluation metrics**, focusing on establishing a technical baseline for language-based RCA capabilities.

B. Data Source and Preprocessing

1) Data Source and Ethics: The dataset consists of **6,582 historical fault reports** sourced from Ericsson's internal **Jira system**. Each report includes structured metadata and unstructured text. In compliance with internal data governance and Data Protection Officer (DPO) approval, all personal identifiers and columns (e.g., author names, comments) were **removed** to ensure anonymity and data security. All experiments were conducted within a secure corporate intranet environment.

2) Feature Selection and Ground Truth: The final dataset utilized four key fields. The expert-written RCA serves as the **Ground Truth (Target Variable)**.

- **Input Features:** Summary, Description, and Priority (Importance Code).
- **Target Feature (Ground Truth):** Answer (the expert-written RCA).

The Answer field follows a standardized, multi-component structure, including the Condition, Description of fault, and Correction (See Table **??**).

Component	Function in RCA
Condition	Description of conditions that trigger the fault.
Description of the fault	Detailed technical explanation of the issue.
Description of the correction	The steps taken or planned to resolve the fault.

TABLE I: **Standardized Answer Structure for RCA (Root Cause Analysis)**

3) Data Cleaning and Wrangling: The raw data underwent several preprocessing steps:

1) **Consolidation:** Fault reports pointing to the same underlying resolution were grouped to prevent treating duplicates as independent training samples.
2) **Null Handling:** Entries lacking a value in the critical **Answer** column were removed.
3) **Partitioning:** The cleaned dataset was randomly partitioned into a **Training set (80%)** and a **Validation set (20%)**, stratified by Priority.

C. Model Architectures and Setup

Three distinct LLM architectures were selected to represent different paradigms in text generation relevant to RCA.

TABLE II: **Summary of Model Architectures**

Model	Type	Rationale for RCA
T5 (Base)	Encoder-Decoder (Seq2Seq)	Unified **Text-to-Text** transformation for structured output.
GPT-2 (Medium)	Autoregressive Decoder	Evaluates flexible, **free-form generative** capabilities.
RAG	Retrieval-Augmented	Evaluates benefit of **external grounding** via historical context.

1) Training Environment: Experiments were executed in a containerized **Kubeflow environment** using Python, the Hugging Face Transformers library, and PyTorch. The hardware setup included 1 GPU (16 GB memory) and ∼30 GB RAM.

2) Model Training and Hyperparameters: The T5 and GPT-2 models were fine-tuned on the Training set. Key hyperparameters included setting the Max Input/Output Sequence Length to **512 tokens**. Training was performed for up to 10 epochs using the AdamW optimizer with **early stopping** based on validation loss to prevent overfitting.

D. Evaluation Metrics and Procedure

Model performance was assessed using a combination of automated Natural Language Generation (NLG) metrics against the expert-written Ground Truth:

- **BLEU:** Measures **lexical fidelity** and precision based on n-gram overlap.
- **ROUGE:** Measures **completeness** and recall, focusing on unigram (`ROUGE-1`) and Longest Common Subsequence (`ROUGE-L`) overlap.
- **BERTScore:** Measures **semantic similarity** by computing the cosine similarity between contextual embeddings, accounting for meaning beyond exact word matches.

The **Evaluation Procedure** involved generating an RCA for every entry in the 20% Validation set and computing the three metric scores (BLEU, ROUGE, BERTScore) against the reserved Ground Truth answers.

IV. RESULTS AND DISCUSSION

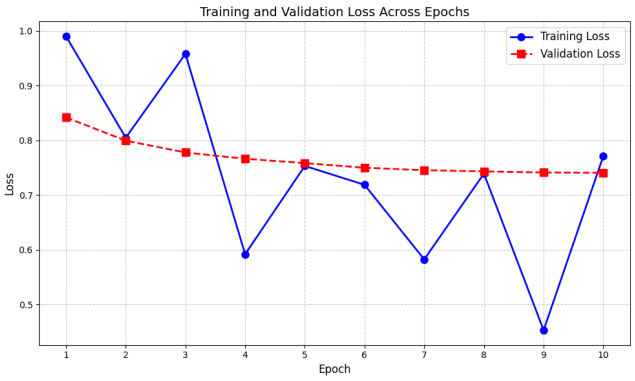
This section presents the quantitative evaluation of the three model architectures: **T5**, **GPT-2**, and **Retrieval-Augmented Generation (RAG)**. Performance is assessed on the 20% validation set using standard Natural Language Generation (NLG) metrics.

A. Training Dynamics Analysis

The fine-tuning process for the generative models (T5 and GPT-2) was monitored to assess stability and generalization.

- **T5 Performance:** The T5 model exhibited minor fluctuations in training loss but maintained a **stable validation loss** throughout all 10 epochs. This stability indicates that the Encoder-Decoder architecture generalized effectively to the unseen fault report data.
- **GPT-2 Performance:** The GPT-2 model showed significant **instability and erratic fluctuation** in training loss (ranging between 1.67 and 3.00). This inconsistent convergence suggests that the autoregressive, decoder-only architecture struggled to adapt to the structured, sequence-to-sequence nature of the RCA task.

The performance of the RAG model is evaluated primarily through its end-to-end output metrics in Section IV-B, as generation loss alone does not fully capture its effectiveness.



(a) T5 Training & Validation Loss

(b) GPT-2 Training & Validation Loss

Fig. 1: T5 and GPT-2 Training and Validation Loss Curves

B. Quantitative Metric Comparison

The models were evaluated by comparing their generated RCA explanations against the expert-written ground truth using lexical overlap metrics (BLEU, ROUGE) and semantic similarity (BERTScore). The results are summarized in Table III.

TABLE III: Model Performance Comparison on RCA Generation

Model	BLEU-4	ROUGE-1	ROUGE-L	ROUGE-2	BERT S
T5	**0.1810**	**0.3869**	**0.3463**	**0.2821**	0.547
GPT-2	0.1210	0.1885	0.1001	0.0432	0.467
RAG	0.0037	0.0682	0.0447	0.0072	**0.771**

1) Lexical and Structural Fidelity: The **T5** model achieved the highest scores across all lexical metrics (BLEU-4, ROUGE-1, ROUGE-L, ROUGE-2). This demonstrates its ability to reproduce the specific terminology and structured format required for formal RCA documentation. The **RAG** model recorded the lowest lexical scores (BLEU-4 = 0.0037), reflecting its tendency to paraphrase retrieved information rather than matching exact n-grams from the ground truth.

2) Semantic Similarity: The **RAG** model achieved the highest semantic similarity score (BERTScore = 0.7715). Since BERTScore evaluates meaning beyond exact word matches, this finding is crucial: the RAG model generates contextually and technically equivalent explanations, highlighting its strength in knowledge grounding. The **T5** model's BERTScore of 0.5478 confirmed a strong balance between structured output and semantic correctness.

C. Synthesis of Findings

The evaluation demonstrates a distinct trade-off in architectural performance for RCA:

- **T5 excels in structured reporting and automation,** achieving the highest lexical scores necessary for outputs that conform to rigid documentation standards.
- **RAG excels in semantic relevance and reasoning,** achieving the highest BERTScore which is critical for providing deep, contextually-grounded insights valuable to a human engineer.

These results provide the quantitative basis for guiding the selection of LLM architectures in practical engineering workflows.

V. CONCLUSION AND FUTURE WORK

REFERENCES

[1] D. F. Pedroso, L. Almeida, L. E. G. Pulcinelli, W. A. A. Aisawa, I. Dutra, and S. M. Bruschi, "Anomaly detection and root cause analysis in cloud-native environments using large language models and bayesian networks," *IEEE Access*, vol. 13, pp. 77 550–77 564, 2025. DOI: 10.1109/ACCESS.2025.3565220.

[2] Y. Wang et al., "Root cause prediction based on bug reports," *arXiv preprint arXiv:2103.02372*, 2021.

[3] X. Chen et al., "Big data oriented root cause identification approach based on pca and svm for product infant failure," *Journal of Intelligent Manufacturing*, 2017.

[4] H. Zhang et al., "Machine learning based root cause analysis for sdn networks," in *Proceedings of IEEE ICNC*, 2022.

[5] A. Jovic et al., "Root cause classification using naïve bayes on software defect reports," *Automatic Control and Computer Sciences*, 2023.

[6] X. Chen et al., "Big data–oriented root cause identification approach based on pca and svm for product infant failure," *Frontiers in Manufacturing Technology*, 2022.

[7] R. Singh et al., "Deep learning for automated anomaly detection in root cause analysis," *arXiv preprint arXiv:2101.07802*, 2021.

[8] L. Zhao et al., "Kgroot: Root cause analysis in microservices using knowledge graphs and graph convolutional networks," *Expert Systems with Applications*, vol. 235, p. 121 155, 2024.

[9] J. Li et al., "Root cause analysis based on lstm and bayesian neural network for time series fault data," *Algorithms*, vol. 18, no. 1, p. 11, 2023.